

Sistema Gerenciador de Filas Bancárias

1. Visão Geral do Projeto

Este documento estabelece as diretrizes arquiteturais e a especificação técnica para a construção de um sistema de gerenciamento de filas bancárias em Python. O escopo foca na implementação rigorosa da legislação brasileira de prioridades, utilizando uma arquitetura baseada em Filas de Múltiplos Níveis (Multilevel Queues) com controle de escalonamento anti-inanição.

O stack tecnológico definido compreende: * Linguagem Core: Python 3.x * Frontend / Interface: Streamlit * Backend / Persistência: Supabase (PostgreSQL) via API REST * Geração de Dados: Faker e Random

2. Regras de Negócio e Hierarquia Legal

O sistema reflete a hierarquia legal vigente no Brasil, dividindo a estrutura de dados em três níveis de precedência estrita:

1. **Prioridade Máxima (Superprioridade):** Indivíduos com 80 anos ou mais (Lei 13.466/2017).
2. **Prioridade Padrão:** Idosos (60-79 anos), gestantes, lactantes, pessoas com crianças de colo, obesos e pessoas com deficiência (Lei 10.048/2000) e Autistas (Lei 12.764/2012).
3. **Fila Convencional:** Clientes não enquadrados nas leis acima.

3. Estruturas de Dados: collections.deque

Para o armazenamento transitório e processamento da fila em memória de forma altamente performática, será utilizada a estrutura deque (Double-Ended Queue) do módulo nativo collections do Python.

3.1 Fundamentação Técnica e Complexidade Computacional

Um deque é implementado em C (no CPython) como uma Lista Duplamente Encadeada com blocos de memória. Diferente de um array contíguo, cada elemento guarda um ponteiro para o anterior e para o próximo. Isso garante complexidade de tempo $O(1)$ (Tempo Constante) para operações nas extremidades. A entrada de um cliente no final da fila ou a chamada do primeiro cliente consome a mesma quantidade mínima de processamento da CPU, independentemente de haver 10 ou 1 milhão de registros.

3.2 Implementação das Funções

A arquitetura exige a instanciação de três filas independentes: * `fila_super = collections.deque()` * `fila_padrao = collections.deque()` * `fila_convencional = collections.deque()`

Métodos utilizados: * Entrada de Cliente: `fila_super.append(dados_cliente)`. Adiciona o cliente ao final da fila da categoria correspondente (garantia do FIFO - First In, First Out). * Chamada de Cliente: `fila_super.popleft()`. Remove e retorna o cliente do início da fila, transferindo-o para o status de "Em Atendimento".

4. Algoritmo de Escalonamento (Anti-Starvation)

Para evitar que a Fila Convencional sofra de Inanição (Starvation) em horários de pico, será implementado um algoritmo de Escalonamento Proporcional (Weighted Round-Robin) gerenciado por contadores.

4.1 Lógica de Contadores

O sistema define um “Quantum” (ciclo de atendimento) fixo. Um padrão mercadológico recomendado é a proporção 3:2:1. * A função gerenciadora avaliará variáveis de estado (count_super, count_padrao). * Passo 1: Tenta extrair até 3 elementos contínuos usando popleft() da Fila Máxima. * Passo 2: Atingido o limite de 3 ou estando a fila vazia, o ponteiro chaveia para a Fila Padrão e tenta extrair até 2 elementos. * Passo 3: Extrai obrigatoriamente 1 elemento da Fila Convencional (se houver). * Passo 4: Os contadores são zerados.

Essa mecânica matemática força a movimentação contínua da fila regular, garantindo o cumprimento dos prazos máximos de espera estabelecidos pelos órgãos de defesa do consumidor, sem ferir a proporção de atendimento prioritário.

5. Integração com Banco de Dados: Supabase (supabase-py)

A persistência do estado das filas será gerida pelo Supabase. O banco PostgreSQL garante propriedades ACID (Atomicidade, Consistência, Isolamento, Durabilidade).

5.1 Modelagem das Tabelas

Tabela clientes_fila: Campos essenciais: id (UUID), nome (Text), nivel_prioridade (Int), motivo (Text), data_chegada (Timestamptz), status (Text). Tabela atendimentos: Campos: id_chamada (UUID), cliente_id (FK), guiche (Int), data_chamada (Timestamptz).

5.2 Funções da Biblioteca supabase-py

O SDK oficial intermediará as requisições REST: * Inserção (Nova Senha): `supabase.table('clientes_fila').insert({'nome': nome, 'nivel_prioridade': nivel, 'status': 'aguardando'}).execute()` * Busca Ordenada (Recuperar o próximo): Para buscar o próximo cliente elegível, o sistema fará um Query Builder: `supabase.table('clientes_fila').select('').eq('nivel_prioridade', nivel_alvo).eq('status', 'aguardando').order('data_chegada', desc=False).limit(1).execute()` Atualização (Marcar como Atendido): `supabase.table('clientes_fila').update({'status': 'atendido'}).eq('id', cliente_id).execute()`

6. Motor de Simulação Estocástica (random e Faker)

O ambiente de testes dependerá de dados sintéticos realistas.

6.1 Parametrização Probabilística

O controle do fluxo de entrada utilizará a função choices da biblioteca nativa random, que permite definir taxas de ocorrência: `* categoria_sorteada = random.choices([0, 1, 2], weights=[0.15, 0.35, 0.50], k=1)[0]`. Isso garante que 15% das simulações sejam Superprioridade, 35% Prioridade Padrão e 50% Convencionais.

6.2 Geração de Payload

Acionada pela categoria sorteada, a biblioteca Faker constrói o perfil demográfico com localização pt_BR. * Instanciação: `fake = Faker('pt_BR')` * Funções de Geração: *

`fake.name()`: Gera nomes autênticos. * `fake.cpf()`: Gera CPFs com formatação e dígito verificador matemático válido. * `fake.date_of_birth(minimum_age=80, maximum_age=105)`: Ao identificar que a categoria sorteada foi “Superprioridade”, esta função assegura que o objeto de data retornado corresponda à exigência legal da idade.

7. Interface de Usuário e Gestão de Estado: Streamlit

O Streamlit construirá o frontend reativo do sistema de atendimento e dos painéis.

7.1 Gestão de Variáveis de Sessão (`st.session_state`)

Como o Streamlit reexecuta o script a cada interação (clique em botões), contadores em memória normal seriam zerados. Utiliza-se o dicionário global da sessão para preservar os contadores do algoritmo Anti-Starvation: * if ‘contador_super’ not in `st.session_state`: `st.session_state[‘contador_super’]` = 0. * Sempre que o guichê chama um cliente, esse valor é incrementado localmente: `st.session_state.contador_super += 1`.

7.2 Funções de Renderização e Layout

Grid e Painéis: `col1, col2 = st.columns(2)` para dividir a tela entre o painel de chamadas e o console de controle do guichê. Tabelas Dinâmicas: A função `st.dataframe(df)` receberá o retorno das queries do Supabase convertidas em DataFrames do Pandas, renderizando a visualização das próximas senhas a serem chamadas com suporte à ordenação visual. Botões de Ação Transacional: if `st.button(‘Chamar Próximo’)`: engatilhará o fluxo: avalia contadores -> consome API do Supabase -> atualiza tabela e sessão.

8. Arquitetura Orientada a Objetos (POO)

A transição deste projeto para o paradigma de Programação Orientada a Objetos (POO) visa garantir manutenibilidade, testabilidade e escalabilidade. O design baseia-se nos princípios SOLID, especificamente o Princípio da Responsabilidade Única (SRP) e o Princípio da Inversão de Dependência (DIP). O código será estritamente modularizado, isolando a lógica de negócios da lógica de interface e de banco de dados. O sistema não tráfegará estruturas de dados primitivas (como dicionários soltos) entre suas camadas; toda informação será encapsulada em objetos com tipagem estrita para garantir a segurança dos contratos.

9. Modelagem de Domínio (Domain-Driven Design)

O domínio central da aplicação mapeia as entidades do mundo real (as regras bancárias e os clientes) para estruturas imutáveis de dados.

9.1 O Padrão de Enumeração Estrita (Tipagem de Prioridade)

Para erradicar a presença de “Números Mágicos” (ex: usar 0, 1 e 2 no meio da lógica), a arquitetura exige a implementação de uma classe de Enumeração (Enum). *

Fundamentação: Isso garante segurança de tipo (Type Safety). O sistema passa a entender explicitamente o que é “Superprioridade” em vez de um inteiro genérico, interceptando falhas em tempo de compilação.

9.2 A Entidade de Transferência de Dados (Data Class)

O cliente será representado por uma classe decorada com atributos de dados puros (Dataclass). * Fundamentação: Abstrai-se o código repetitivo de inicialização. * Contrato de Responsabilidade: Esta classe representa apenas a estrutura da informação (Nome, CPF,

Categoria). Foi descartado o padrão Active Record para respeitar a Responsabilidade Única (o cliente existe na memória, mas não opera sobre a infraestrutura do banco).

10. Padrões de Projeto Estruturais e Criacionais

A complexidade da simulação e da persistência de dados exige a adoção de padrões de projeto (Design Patterns) consagrados.

10.1 O Padrão Factory (Fábrica de Mocking Estocástico)

A geração de dados realistas de teste acopla lógicas de probabilidade à injeção demográfica. * Arquitetura: Será construída uma classe “Fábrica”. A interface não saberá como um cliente é criado; apenas solicitará uma instância. * Mecânica Interna: A Fábrica rodará o motor probabilístico (random) e instanciará o Faker para forjar os atributos condizentes com a categoria sorteada.

10.2 O Padrão Repository (Camada de Persistência Supabase)

As regras de fila não podem conter chamadas diretas para o Supabase. * Arquitetura: Implementar uma classe Repositório para atuar como uma fachada (Facade). * Fundamentação: Esta classe encapsula as credenciais e executa as inserções e buscas. Se o projeto migrar do Supabase para outro SGBD, apenas os métodos deste Repositório sofrerão alterações, preservando 100% da regra de negócios.

11. O Motor de Negócios (Core Queue Manager)

O coração do sistema é uma classe gerenciadora que orquestra as Filas de Múltiplos Níveis e o mecanismo Anti-Inanição.

11.1 Encapsulamento Restrito e Injeção de Dependência

- Injeção do Repositório: O Gerenciador receberá a instância do Banco de Dados via construtor, permitindo testes unitários com repositórios “falsos”. * Proteção de Estado: Os ponteiros das três filas (deque) e os contadores numéricos (regra 3:2:1) devem ser declarados como atributos privados. * Risco: Expor contadores publicamente permitiria que a camada visual alterasse o algoritmo inadvertidamente, quebrando o cumprimento legal.

11.2 Comportamento Operacional

O Gerenciador atua em três frentes: * Sincronização: Busca o estado salvo no banco e repovoa as filas em memória durante a inicialização. * Inserção O(1): Acopla a entrada de um cliente ao final de seu deque e notifica o Repositório. * Algoritmo Extrator: Avalia os limites dos contadores privados e as condições das filas para extrair o cliente exato (popleft), garantindo o ciclo.

12. Arquitetura de Estado no Frontend (Streamlit)

O modelo de execução do Streamlit destrói e reexecuta todas as variáveis locais a cada interação do usuário.

12.1 O Desafio e a Solução Singleton

Implementar a POO no Streamlit exige adaptação. Se o Gerenciador for instanciado convencionalmente, os contadores serão zerados a cada clique. * Arquitetura: As instâncias globais (Repositório, Fábrica e Gerenciador) devem ser injetadas no Dicionário de Estado de

Sessão (session_state) na primeira execução. * Fundamentação: Isso simula o Padrão Singleton, garantindo que a interface opere sobre o exato mesmo objeto continuamente.

13. Fluxo de Trabalho Coworking e Estratégia de Versionamento (Git)

Para assegurar o paralelismo no desenvolvimento e minimizar conflitos, a equipe adotará a estratégia Git Flow simplificado.

13.1 A Estrutura de Branches (Ramificações)

O repositório no GitHub obedecerá à seguinte hierarquia: * main: Código estável e homologado. * develop: Ramo base de integração. Contém a versão mais recente em estabilização. * feature/: Ramos temporários para o trabalho diário.

13.2 O Ciclo de Desenvolvimento Diário (Passo a Passo)

1. Sincronização Matinal: O desenvolvedor atualiza seu ambiente rastreando o código integrador (git checkout develop -> git pull).
2. Isolamento do Escopo: Cria uma ramificação limpa focada em apenas uma responsabilidade (ex: git checkout -b feature/modelagem).
3. Commits Semânticos: O trabalho é salvo localmente em pequenos pacotes lógicos, evitando misturar frontend e backend no mesmo commit.
4. Code Review (Pull Request): O ramo é enviado ao GitHub. Um Pull Request (PR) é aberto apontando para develop. O código passa por revisão da equipe e, se aprovado, é mesclado (merged).

Este rigor arquitetural permite que desenvolvedores atuem em paralelo (ex: um no Repositório e outro na Fábrica) sem jamais causarem conflitos de sobreposição no código.